Optimisation: gradients, gradient descent and its stochastic variants

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Training a model means minimising an objective over its parameters. This note reviews the calculus needed — partial derivatives, gradients, the Hessian — and develops gradient descent: the role of the learning rate and of conditioning, convex and non-convex landscapes, and the mini-batch stochastic gradients that make training on large datasets possible. Momentum, Newton's method and Adam complete the toolbox used to train every neural network, including the large language models whose training recipes close the note.

1 Learning is optimisation

The training phase of most algorithms in this course solves

$$\theta^* = \arg\min_{\theta \in \mathbb{R}^p} J(\theta), \qquad J(\theta) = \frac{1}{n}\sum_{i=1}^{n} L\big(f_\theta(\mathbf{x}_i), y_i\big) + \lambda\,\Omega(\theta),$$

the regularised empirical risk of Note 4, Section 7, as a function of the p parameters. Minimising the sum or the mean is the same problem; the mean keeps the scale independent of n. Picture J as a landscape over parameter space: training means finding a low point.

2 Derivatives, gradients and curvature

For $J : \mathbb{R}^p \to \mathbb{R}$, the partial derivative $\partial J/\partial\theta_k$ measures the sensitivity of J to a change of θ_k alone, the others held fixed. The *gradient* collects them,

$$\nabla J(\theta) = \Big(\frac{\partial J}{\partial \theta_1}(\theta), \dots, \frac{\partial J}{\partial \theta_p}(\theta)\Big)^{\mathsf{T}} \in \mathbb{R}^p,$$

and the matrix of second derivatives is the *Hessian*, $H_{kl} = \partial^2 J/\partial\theta_k\partial\theta_l$. Taylor's theorem summarises what they mean:

$$J(\theta + \boldsymbol{\delta}) = J(\theta) + \nabla J(\theta)^{\mathsf{T}}\boldsymbol{\delta} + \tfrac{1}{2}\boldsymbol{\delta}^{\mathsf{T}} H(\theta)\,\boldsymbol{\delta} + O(\|\boldsymbol{\delta}\|^3). \tag{1}$$

Among all small steps of fixed length, the first-order term is most negative for $\boldsymbol{\delta}$ pointing along $-\nabla J$: the gradient points uphill, in the direction of steepest ascent, and is perpendicular to the contour lines of J. At a minimum the slope vanishes, $\nabla J(\theta^*) = \mathbf{0}$. A point with zero gradient is a local minimum if H is positive definite, a local maximum if it is negative definite and a *saddle point* if H has eigenvalues of both signs.

Closed form or search. Sometimes $\nabla J = \mathbf{0}$ can be solved analytically: the maximum-likelihood Gaussian (Note 2, Section 6) and linear and ridge regression (Note 8) are examples. Usually it cannot,

and we must search. Trying all values on a grid is hopeless beyond a handful of parameters — ten values per parameter and $p = 20$ already give 10^{20} evaluations, the curse of dimensionality of Note 3 once more — so we follow the local slope instead.

3 Gradient descent

Starting from an initial θ_0 (random, or by a heuristic suited to the model), repeat

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t), \tag{2}$$

where the *learning rate* (step size) $\eta > 0$ is a hyperparameter. Stop when the updates no longer change θ appreciably, for instance when $\|\nabla J(\theta_t)\| < \epsilon$, or earlier when the validation error stops improving (early stopping, Note 4, Section 7).

What the learning rate does. Near a minimum, (1) says J is approximately quadratic, $J(\theta) \approx J^* + \frac{1}{2}(\theta - \theta^*)^{\mathsf{T}} A (\theta - \theta^*)$ with A the Hessian at θ^*. Then (2) becomes $\theta_{t+1} - \theta^* = (I - \eta A)(\theta_t - \theta^*)$, and along each eigenvector of A with eigenvalue λ_i the error is multiplied by $1 - \eta\lambda_i$ at every step. Three consequences follow.

- Gradient descent converges only if $|1 - \eta\lambda_i| < 1$ for every i, i.e. $\eta < 2/\lambda_{\max}$; a larger step makes the steepest direction oscillate with growing amplitude.
- The flattest direction shrinks by $1 - \eta\lambda_{\min}$ per step, slow if $\lambda_{\min} \ll \lambda_{\max}$.
- With the best fixed step, the error decreases by a factor $(\kappa - 1)/(\kappa + 1)$ per step, where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the *condition number*. Elongated valleys are hard.

Figure 1 shows a valley with $\kappa = 25$. With $\eta = 0.01$ the descent is stable but crawls; with $\eta = 0.075$, close to the limit $2/25 = 0.08$, it zigzags across the valley and makes steady progress. This is why features are standardised before training (Note 5, Section 5): very different feature scales stretch the valley.

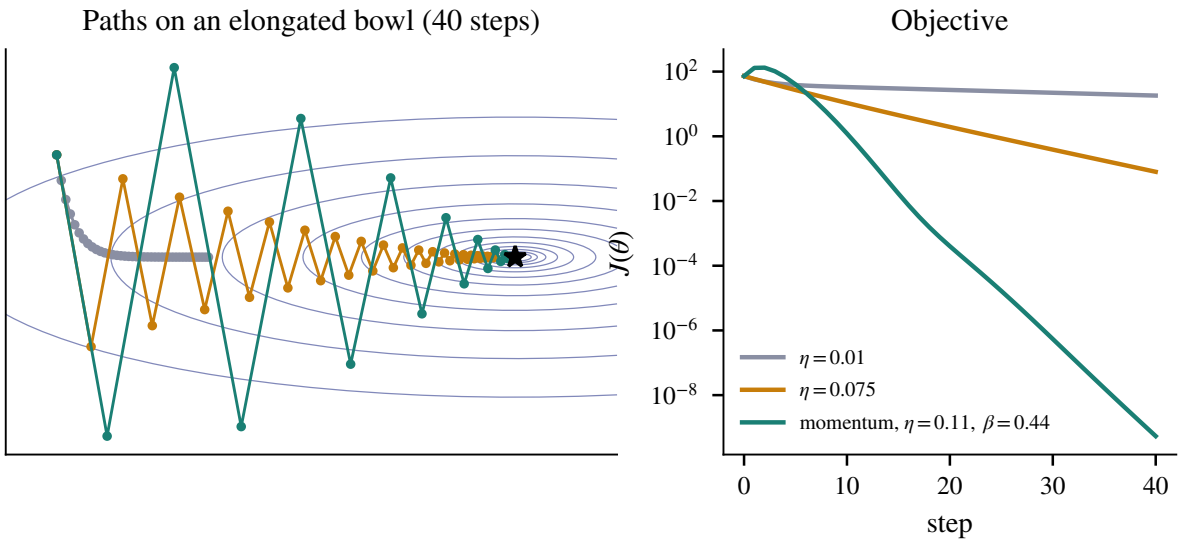


Figure 1. Forty steps on $J(\theta) = \frac{1}{2}(\theta_1^2 + 25\,\theta_2^2)$ from the same start. After 40 steps $J = 18$ for $\eta = 0.01$, 0.08 for $\eta = 0.075$, and 5×10^{-10} with momentum (Section 6).

Schedules. A fixed step is rarely ideal: large steps make fast early progress, small steps settle into the minimum. A classical schedule is $\eta_t = \eta_0\, t_0/(t_0 + t)$, with η_0 and t_0 hyperparameters. Modern deep-learning practice uses a short linear *warm-up* from a small value, which avoids instabilities while the parameters are still random, followed by a slow decay, often along a half cosine.

4 Convex and non-convex objectives

A function is *convex* if the chord between any two points of its graph lies above the graph; for twice differentiable J, if H is positive semi-definite everywhere. Every local minimum of a convex function is global, so gradient descent with a suitable step reaches the global minimum from any start. Least squares, ridge regression, logistic regression and the linear support vector machine (Notes 8 and 9) all have convex objectives.

Neural networks do not. Their landscapes have many stationary points, and where gradient descent ends depends on where it starts (Figure 2). In high dimension the picture is less bleak than this one-dimensional cartoon suggests: most stationary points with high loss are saddle points rather than poor local minima, because it is unlikely that all p curvatures are positive at once, and the minima that training finds tend to have similar loss [4]. Slow progress near saddles, rather than being trapped, is the more common difficulty.

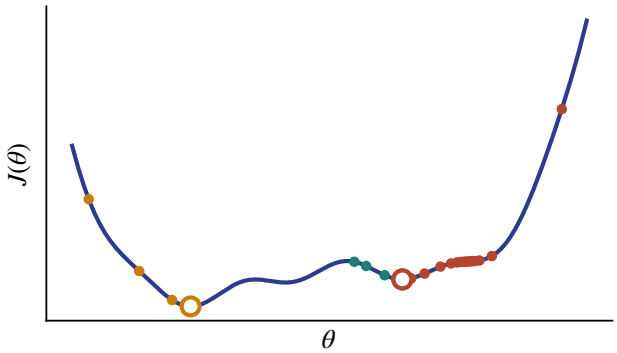


Figure 2. Gradient descent from three starting points on a non-convex function ends in three different minima (open circles).

5 Stochastic gradients

The gradient of a mean is the mean of the gradients:

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta L\big(f_\theta(\mathbf{x}_i), y_i\big) \quad (+\,\lambda \nabla \Omega).$$

Batch gradient descent computes all n terms before every update, which costs a full pass over the data per step. *Mini-batch* gradient descent replaces the mean over D_{train} by the mean over a random subset B of $n' \ll n$ examples:

$$\mathbf{g}_B(\theta) = \frac{1}{n'}\sum_{i\in B} \nabla_\theta L\big(f_\theta(\mathbf{x}_i), y_i\big), \qquad \theta \leftarrow \theta - \eta\, \mathbf{g}_B(\theta).$$

The case $n' = 1$ is *stochastic* (online) gradient descent, SGD; $n' = n$ recovers the batch method. If B is drawn at random, $\mathbf{g}_B$ is an unbiased estimate of ∇J whose variance falls like $1/n'$. In practice the data are shuffled once per *epoch* (one pass over D_{train}) and cut into consecutive mini-batches.

Figure 3 compares the three on logistic regression with $n = 2000$. After one epoch, batch gradient descent has taken one step; mini-batch descent with $n' = 32$ has taken 63 and is already within 0.013 of the optimal loss. With a constant step, however, the stochastic methods do not converge: they fluctuate in a band whose width is set by the gradient noise and the learning rate. Pure SGD, the noisiest, ends further from the optimum after 20 epochs than batch descent. Mini-batches are the compromise used in practice, and they are also what parallel hardware wants: the n' gradients are computed together as matrix products on a GPU, so a mini-batch of 256 costs little more time than a single example. Decreasing the learning rate removes the noise floor at the end of training.

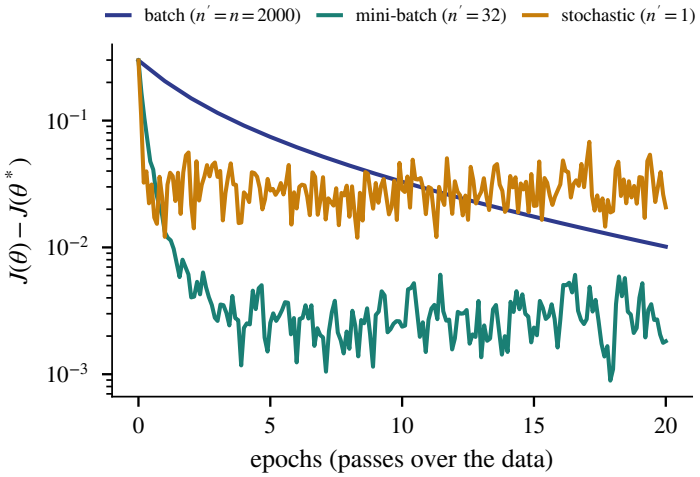


Figure 3. Excess training loss of logistic regression ($d = 20$, $n = 2000$) against the number of passes over the data, with constant learning rates. Mini-batch descent reaches 0.0018 after 20 epochs; batch descent 0.010; SGD stays around 0.02.

6 Beyond plain gradient descent

Momentum. Accumulate a velocity and move with it, $\mathbf{v}_{t+1} = \beta\,\mathbf{v}_t - \eta \nabla J(\theta_t)$, $\theta_{t+1} = \theta_t + \mathbf{v}_{t+1}$, with $0 \le \beta < 1$. Components of the gradient that keep their sign add up; those that alternate, as across a valley, cancel. With well-chosen η and β the convergence factor on a quadratic improves from $(\kappa - 1)/(\kappa + 1)$ to $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ [8]: in Figure 1 momentum reaches $J \approx 10^{-9}$ in the same forty steps.

Newton's method. Minimise the quadratic model (1) exactly: $\theta \leftarrow \theta - H^{-1}\nabla J$. On a quadratic objective it converges in one step whatever the condition number, and near a minimum it converges quadratically. But forming and solving with the $p \times p$ Hessian costs $O(p^2)$ memory and $O(p^3)$ time, it is unreliable far from a minimum or near saddles, and it does not combine well with noisy mini-batch gradients. Quasi-Newton methods such as L-BFGS build a low-rank curvature estimate from successive gradients and are excellent for medium-sized, deterministic problems; logistic regression is often trained with Newton's method itself.

Adaptive methods. Adam keeps exponential moving averages of the gradient and of its element-wise square, and divides the first by the square root of the second [6]. Each parameter thus gets its own step size, which compensates for very different gradient scales across a network. AdamW applies weight decay directly to the parameters instead of through the gradient, which matters for adaptive methods [7]; it is the default optimiser for transformers.

Where the gradients come from. For a neural network with millions of parameters, ∇J is computed by reverse-mode *automatic differentiation* — backpropagation — at a cost of a small constant times one evaluation of J, exactly, not by finite differences [1]. This is the subject of the rest of week 4.

Constraints. When parameters must satisfy constraints (non-negative, summing to one, inside a ball), one can project onto the feasible set after each step, reparameterise (a softmax always sums to one) or use Lagrange multipliers. Linear and quadratic programmes form a well-understood class; the support vector machine of Note 9 is a quadratic programme.

In physics — optimisers as mechanics

Gradient descent with small steps follows the *gradient flow* $\dot{\theta} = -\nabla J(\theta)$: overdamped motion of a particle in the potential J. Momentum restores inertia — Polyak's "heavy ball", $m\ddot{\theta} + \gamma\dot{\theta} = -\nabla J$, discretised — which is why it rolls through narrow valleys instead of zigzagging. The noise of mini-batch gradients acts like a temperature: SGD resembles Langevin dynamics, and adding properly scaled Gaussian noise to the updates turns it into a sampler of the posterior distribution of the parameters [9].

Modern connection — how a large language model is trained

The recipe for GPT-3, typical of large models, uses every idea above [3]: Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, the gradient norm clipped at 1.0 to tame rare large steps, a linear warm-up over the first 375 million tokens followed by cosine decay of the learning rate, weight decay of 0.1, and mini-batches that grow to about 3.2 million tokens. The objective is the next-token cross-entropy (Note 2, Section 2), and each update is a mini-batch gradient step of Section 5, on 1.75×10^{11} parameters.

Summary The gradient points uphill and vanishes at minima; the Hessian describes curvature. Gradient descent steps against the gradient with learning rate $\eta < 2/\lambda_{\max}$, and its speed is limited by the condition number. Convex objectives have a single basin; neural networks do not, but saddles rather than bad minima are the usual obstacle. Mini-batch stochastic gradients trade exactness for many cheap steps and leave a noise floor that a decaying learning rate removes. Momentum, Newton and Adam reshape the steps; automatic differentiation supplies the gradients.

Exercises

E1. For $J(\theta) = \frac{1}{2}\theta^{\mathsf{T}} A\theta - \mathbf{b}^{\mathsf{T}}\theta$ with A symmetric positive definite, find ∇J, H and θ^*. Show that gradient descent with $\eta = 2/(\lambda_{\max} + \lambda_{\min})$ reduces the error by $(\kappa - 1)/(\kappa + 1)$ per step, and that Newton's method converges in one step.

E2. Rescale the second coordinate of the problem in Figure 1 by a factor 5. What is the new condition number, and what does this say about standardising features?

E3. Show that the mini-batch gradient is unbiased and that its covariance is $\frac{1}{n'}$ times that of a single-example gradient when the batch is drawn with replacement.

E4. Implement batch, mini-batch and stochastic gradient descent for logistic regression and reproduce Figure 3. Then use a decaying learning rate $\eta_t = \eta_0/(1 + t/t_0)$ for SGD and report the

excess loss after 20 epochs.

E5. Find the stationary points of $J(\theta_1, \theta_2) = \theta_1^2 - \theta_2^2 + \tfrac{1}{2}\theta_2^4$ and classify them with the Hessian. Starting at $(1, 0)$, where does gradient descent go? And starting at $(1, 10^{-6})$?

E6. Estimate the memory needed to store the Hessian of a model with $p = 10^8$ parameters in 32-bit floats, and compare with storing the parameters and the two moment estimates of Adam.

Further reading. Goodfellow, Bengio and Courville [5], chapters 4 and 8; Bottou, Curtis and Nocedal [2] for stochastic optimisation in machine learning.

References

[1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* 18(153), 1–43 (2018).

[2] L. Bottou, F. E. Curtis and J. Nocedal, Optimization methods for large-scale machine learning, *SIAM Rev.* 60, 223–311 (2018).

[3] T. Brown *et al.*, Language models are few-shot learners, *NeurIPS* (2020).

[4] Y. N. Dauphin *et al.*, Identifying and attacking the saddle point problem in high-dimensional non-convex optimization, *NeurIPS* (2014).

[5] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[6] D. P. Kingma and J. Ba, Adam: a method for stochastic optimization, *ICLR* (2015).

[7] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, *ICLR* (2019).

[8] B. T. Polyak, Some methods of speeding up the convergence of iteration methods, *USSR Comput. Math. Math. Phys.* 4(5), 1–17 (1964).

[9] M. Welling and Y. W. Teh, Bayesian learning via stochastic gradient Langevin dynamics, *ICML* (2011).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.